IFPB - Campus Campina Grande
Programação Orientada a Objetos - Unidade 02

Nome: _____ Nota: _____

O código do jogo está ao final da prova. As questões são independentes; classes e alterações apresentadas valem apenas na própria questão.

Todos os tipos públicos estão em arquivos separados, no mesmo projeto, sem `package`. Considere disponíveis `List`, `ArrayList` e, nos testes, `assertEquals` do JUnit. Os argumentos são não nulos, os índices são válidos e os valores iniciais respeitam os limites dos objetos.

Q1. (6 pontos) Escreva a saída do trecho abaixo.

```
Personagem p = new Personagem(88);
p.recuperarVida(19);
p.receberDano(-8);
System.out.println(p.getVida());
p.receberDano(130);
p.recuperarVida(7);
p.recuperarVida(-5);
System.out.println(p.getVida());
```

a) _____

b) _____

Q2. (3 pontos) Escreva a saída do trecho abaixo.

```
Equipamento item = new Amuleto(0);
item.recarregar(5);
item.usar(new Personagem(61));
System.out.println(item.descrever());
```

Q3. (6 pontos) Escreva a saída do trecho abaixo.

```
Inventario inventario = new Inventario();
Pocao pocao = new Pocao();
inventario.adicionar(new Cajado(2));
inventario.adicionar(pocao);
pocao.usar(new Personagem(36));
inventario.listar();
```

a) _____

b) _____

Q4. (6 pontos) Escreva a saída do trecho abaixo.

```
Cajado cajado = new Cajado(1);
Inventario inventario = new Inventario();
inventario.adicionar(cajado);
inventario.adicionar(cajado);
Personagem alvo = new Personagem(40);
Usavel primeiro = (Usavel) inventario.obter(0);
Usavel segundo = (Usavel) inventario.obter(1);
primeiro.usar(alvo);
System.out.println(segundo.usar(alvo));
System.out.println(alvo.getVida());
```

a) _____

b) _____

Q5. (3 pontos) Escreva a saída do trecho abaixo.

```
Personagem usuario = new Personagem(74);
Personagem alvo = new Personagem(50);
Usavel escolha = new Cajado(1);
usuario.equipar(escolha);
escolha = new Amuleto(2);
usuario.usarEquipadoEm(alvo);
System.out.println(alvo.getVida());
```

Q6. (6 pontos) Escreva a saída do trecho abaixo.

```
Personagem a = new Personagem(32);
Personagem b = new Personagem(68);
a.equipar(new Cajado(1));
b.equipar(new Amuleto(1));
a.usarEquipadoEm(b);
b.usarEquipadoEm(a);
System.out.println(a.getVida());
System.out.println(b.getVida());
```

a) _____

b) _____

Q7. (3 pontos) Escreva a saída produzida pelo trecho abaixo.

```
Item item = new Cajado(1);
Usavel uso = (Usavel) item;
Recarregavel recarga = (Recarregavel) item;
System.out.println(uso == recarga);
```

Q8. (6 pontos) Escreva a saída produzida pelo trecho abaixo.

```
Personagem alvo = new Personagem(100);
Amuleto item = new Amuleto(1);
System.out.println(item.usar(alvo));
System.out.println(item.getCargas());
```

a) _____

b) _____

Q9. (6 pontos) Escreva a saída produzida pelo trecho abaixo.

```
Personagem p = new Personagem(48);
System.out.println(p.usarEquipadoEm(p));
p.equipar(new Pocao());
if (p.usarEquipadoEm(p)) {
    p.receberDano(6);
}
if (p.usarEquipadoEm(p)) {
    p.receberDano(6);
}
System.out.println(p.getVida());
```

a) _____

b) _____

Q10. (3 pontos) Considere a classe `CajadoLivre`, apresentada apenas nesta questão. Escreva a saída do trecho que a utiliza.

```
public class CajadoLivre extends Cajado {
    public CajadoLivre() {
        super(0);
    }

    @Override
    public boolean usar(Personagem alvo) {
        alvo.receberDano(5);
        return true;
    }
}
```

```
Equipamento item = new CajadoLivre();
Personagem alvo = new Personagem(53);
item.usar(alvo);
System.out.println(alvo.getVida());
```

_____

Q11. (6 pontos) Escreva a saída produzida pelo trecho abaixo.

```
Inventario inventario = new Inventario();
Amuleto amuleto = new Amuleto(0);
inventario.adicionar(new Pocao());
inventario.adicionar(amuleto);
inventario.adicionar(new Cajado(1));
int recarregados = 0;
for (int i = 0; i < 3; i++) {
    Item item = inventario.obter(i);
    if (item instanceof Recarregavel) {
        Recarregavel r = (Recarregavel) item;
        r.recarregar(1);
        recarregados++;
    }
}
System.out.println(recarregados);
System.out.println(amuleto.getCargas());
```

a) _____

b) _____

Q12. (6 pontos) Para acrescentar uma recuperação de vida ao uso de um item, foi criada a classe `UsavelComCura`, válida apenas nesta questão. Escreva a saída do trecho que a utiliza.

```
public class UsavelComCura implements Usavel {
    private Usavel base;

    public UsavelComCura(Usavel base) {
        this.base = base;
    }

    @Override
    public boolean usar(Personagem alvo) {
        if (!base.usar(alvo)) {
            return false;
        }
        alvo.recuperarVida(5);
        return true;
    }
}
```

```
Personagem alvo = new Personagem(98);
Usavel uso = new UsavelComCura(
    new UsavelComCura(new Cajado(1)));
uso.usar(alvo);
System.out.println(alvo.getVida());
uso.usar(alvo);
System.out.println(alvo.getVida());
```

a) _____

b) _____

Q13. (5 pontos) Uma desenvolvedora tenta mostrar a descrição do item que será usado:

```
Usavel escolha = new Amuleto(1);
System.out.println(escolha.descrever());
```

O compilador rejeita a chamada a `descrever`. Qual explicação está correta?

A) `Amuleto` precisa declarar novamente `descrever`, pois métodos herdados não podem ser chamados por clientes.

B) O tipo `Usavel` não oferece `descrever`, embora o objeto guardado em `escolha` possua esse método.

C) O método `descrever` de `Equipamento` não pode ser usado porque essa classe é abstrata.

D) Uma variável declarada com uma interface não pode receber um objeto de uma classe concreta.

E) A chamada só poderia ser verificada durante a execução, quando o objeto de `escolha` fosse conhecido.

Q14. (5 pontos) Na próxima versão do jogo, cada personagem deverá guardar e consultar seus itens. Durante a implementação, uma desenvolvedora apresenta esta proposta:

"A classe `Inventario` já tem os métodos `adicionar`, `obter` e `listar`. Podemos fazer `Personagem` herdar de `Inventario` para aproveitar esses métodos."

```
public class Personagem extends Inventario {
    // Campos e métodos do personagem.
}
```

Você está revisando essa proposta. Qual comentário orienta melhor a decisão de projeto?

A) "A herança é adequada: se o personagem precisa desses métodos, isso é suficiente para defini-lo como uma especialização de inventário."

B) "A herança não permite esse reaproveitamento: como a lista de itens é privada, os métodos herdados não conseguem acessá-la."

C) "Seria melhor colocar um inventário dentro do personagem, embora isso exija copiar para `Personagem` a implementação dos três métodos."

D) "Podemos manter a herança e sobrescrever os três métodos: assim, o personagem aproveita o código sem ser tratado como um inventário."

E) "A herança permite o reaproveitamento, mas torna o personagem um tipo de inventário. Ele poderia ter um inventário e usar os métodos desse objeto."

Q15. (5 pontos) Atualmente, `Personagem.equipar` recebe um `Usavel`. Um desenvolvedor propõe trocar o tipo do parâmetro para `Equipamento`, mantendo o campo `equipado` e o corpo do método:

```
public void equipar(Equipamento item) {
    equipado = item;
}
```

Ele justifica a mudança dizendo que o personagem precisa de um objeto com o método `usar`. Qual avaliação está correta?

A) A mudança não afeta as chamadas existentes, pois o tipo do parâmetro apenas documenta o argumento recebido.

B) O método deixa de compilar, pois um campo do tipo `Usavel` não pode guardar uma referência recebida como `Equipamento`.

C) O método continua aceitando poções, pois `Pocao` possui a mesma assinatura de `usar` que os equipamentos.

D) Chamadas com poções deixam de compilar, embora `Usavel` já ofereça a operação de que o personagem precisa.

E) A mudança faz todo item equipado executar `Equipamento.usar`, mesmo quando sua classe sobrescreve esse método.

Q16. (5 pontos) Durante uma revisão, foi proposta esta nova implementação de `Equipamento.recarregar`:

```
public void recarregar(int quantidade) {
    if (quantidade <= 0) {
        return;
    }
    cargas = quantidade;
    if (cargas > maximo) {
        cargas = maximo;
    }
}
```

Qual teste passa na implementação original e falha na proposta? Considere cada alternativa independentemente.

A)
```
Cajado item = new Cajado(0);
item.recarregar(1);
assertEquals(1, item.getCargas());
```

B)
```
Cajado item = new Cajado(2);
item.recarregar(3);
assertEquals(3, item.getCargas());
```

C)
```
Cajado item = new Cajado(1);
item.recarregar(1);
assertEquals(2, item.getCargas());
```

D)
```
Cajado item = new Cajado(3);
item.recarregar(5);
assertEquals(3, item.getCargas());
```

E)
```
Cajado item = new Cajado(2);
item.recarregar(0);
assertEquals(2, item.getCargas());
```

Q17. (5 pontos) A equipe decidiu acrescentar uma restrição ao uso dos equipamentos: se o alvo estiver com vida zero, a tentativa deverá retornar `false`, sem alterar sua vida nem gastar cargas. Essa regra ainda não está implementada. Ela deverá valer tanto pelo uso via personagem quanto por uma chamada direta a `equipamento.usar(alvo)`.

Onde inserir a verificação `alvo.getVida() == 0` para cumprir essa regra?

A) Em `Personagem.recuperarVida`, retornando antes de alterar a vida.

B) Em `Equipamento.usar`, retornando `false` antes de chamar `aplicarEm` e de diminuir as cargas.

C) Em `Equipamento.usar`, retornando `false` depois de chamar `aplicarEm`, antes de diminuir as cargas.

D) Em `Equipamento.aplicarEm`, retornando antes de realizar o efeito, sem mudar `Equipamento.usar`.

E) Em `Personagem.usarEquipadoEm`, retornando `false` antes de delegar, sem mudar os equipamentos.

Q18. (5 pontos) A equipe preparou dois testes para o uso do cajado. Cada teste começa com seus próprios objetos novos:

```
Cajado item = new Cajado(1);
Personagem alvo = new Personagem(46);
```

Teste I:

```
assertEquals(true, item.usar(alvo));
```

Teste II:

```
item.usar(alvo);
assertEquals(34, alvo.getVida());
```

Qual comportamento incorreto de `usar` seria identificado somente pelo Teste II?

A) Consumir uma carga e retornar `true`, sem causar dano ao alvo.
B) Consumir uma carga, causar 12 pontos de dano e retornar `false`.
C) Retornar `false` sem alterar o alvo nem consumir uma carga.
D) Causar 12 pontos de dano e retornar `true`, sem consumir uma carga.
E) Consumir uma carga, causar 24 pontos de dano e retornar `false`.

Q19. (5 pontos) Ao revisar `Equipamento`, uma desenvolvedora propõe substituir seu método abstrato por:

```
public int getVariacaoVida() {
    return 0;
}
```

`Equipamento` continua abstrata e seus outros métodos são mantidos. Com essa mudança, ela acrescenta:

```
public class Orbe extends Equipamento {
    public Orbe() {
        super("Orbe", 2, 4);
    }
}
```

Qual consequência da proposta está correta?

A) `Orbe` não pode ser instanciada, pois toda subclasse de uma classe abstrata também é abstrata.
B) `Orbe` pode ser instanciada sem sobrescrever `getVariacaoVida`; `Cajado` e `Amuleto` mantêm suas próprias variações.
C) `Orbe` não compila, pois precisa declarar diretamente todos os métodos das interfaces de `Equipamento`.
D) `Orbe` precisa declarar novamente os campos `nome`, `cargas` e `maximo` para utilizar os métodos herdados.
E) `Cajado` e `Amuleto` também passam a usar a variação zero, pois o método concreto da base substitui suas sobrescritas.

Q20. (5 pontos) O inventário passará a guardar no máximo três itens. O método `Inventario.adicionar` já foi alterado para impedir novas inclusões ao atingir esse limite. A equipe propõe acrescentar este método:

```
public List<Item> getItens() {
    return itens;
}
```

Com esse método, outro código consegue ultrapassar o limite de três itens? Por quê?

A) Não. Como o campo `itens` é privado, a lista devolvida só pode ser modificada pelos métodos de `Inventario`.
B) Não. Ao devolver a lista, o método cria uma cópia; alterações nessa cópia não afetam o inventário.
C) Sim. Outro código pode chamar `add` na lista devolvida, sem executar a verificação de `Inventario.adicionar`.
D) Sim. A referência devolvida permite substituir o campo privado `itens` por outra lista criada fora do inventário.
E) Não. Sem um método `setItens`, outro código pode consultar a lista devolvida, mas não acrescentar elementos.

CENÁRIO — CONTRATOS, INVENTÁRIO E PERSONAGEM

Item.java

```java
public interface Item {
    String descrever();
}
```

Usavel.java

```java
public interface Usavel {
    boolean usar(Personagem alvo);
}
```

Recarregavel.java

```java
public interface Recarregavel {
    void recarregar(int quantidade);
    int getCargas();
}
```

Inventario.java

```java
import java.util.ArrayList;
import java.util.List;

public class Inventario {
    private List<Item> itens;

    public Inventario() {
        itens = new ArrayList<>();
    }

    public void adicionar(Item item) {
        itens.add(item);
    }

    public Item obter(int indice) {
        return itens.get(indice);
    }

    public void listar() {
        for (Item item : itens) {
            System.out.println(
                item.descrever());
        }
    }
}
```

Personagem.java

```java
public class Personagem {
    private int vida;
    private Usavel equipado;

    public Personagem(int vidaInicial) {
        vida = vidaInicial;
        equipado = null;
    }

    public void receberDano(int quantidade) {
        if (quantidade > 0) {
            vida -= quantidade;
            if (vida < 0) {
                vida = 0;
            }
        }
    }

    public void recuperarVida(int quantidade) {
        if (quantidade > 0) {
            vida += quantidade;
            if (vida > 100) {
                vida = 100;
            }
        }
    }

    public int getVida() { return vida; }

    public void equipar(Usavel item) {
        equipado = item;
    }

    public Usavel getEquipado() {
        return equipado;
    }

    public boolean usarEquipadoEm(
            Personagem alvo) {
        if (equipado == null) {
            return false;
        }
        return equipado.usar(alvo);
    }
}
```

CENÁRIO — EQUIPAMENTOS E POÇÃO

Equipamento.java

```java
public abstract class Equipamento
        implements Item, Usavel, Recarregavel {
    private String nome;
    private int cargas;
    private int maximo;

    protected Equipamento(String nome,
                          int cargasIniciais,
                          int maximo) {
        this.nome = nome;
        cargas = cargasIniciais;
        this.maximo = maximo;
    }

    @Override
    public boolean usar(Personagem alvo) {
        if (cargas == 0) {
            return false;
        }
        aplicarEm(alvo);
        cargas--;
        return true;
    }

    @Override
    public void recarregar(int quantidade) {
        if (quantidade <= 0) {
            return;
        }
        cargas += quantidade;
        if (cargas > maximo) {
            cargas = maximo;
        }
    }

    @Override
    public int getCargas() { return cargas; }

    @Override
    public String descrever() {
        return nome + ": " + cargas
            + "/" + maximo;
    }

    private void aplicarEm(Personagem alvo) {
        int variacao = getVariacaoVida();
        if (variacao < 0) {
            alvo.receberDano(-variacao);
        } else {
            alvo.recuperarVida(variacao);
        }
    }

    // Variação de vida aplicada por usar:
    // negativa: dano; positiva: recuperação.
    public abstract int getVariacaoVida();
}
```

Cajado.java

```java
public class Cajado extends Equipamento {
    public Cajado(int cargasIniciais) {
        super("Cajado", cargasIniciais, 3);
    }

    @Override
    public int getVariacaoVida() {
        return -12;
    }
}
```

Amuleto.java

```java
public class Amuleto extends Equipamento {
    public Amuleto(int cargasIniciais) {
        super("Amuleto", cargasIniciais, 2);
    }

    @Override
    public int getVariacaoVida() {
        return 15;
    }
}
```

Pocao.java

```java
public class Pocao implements Item, Usavel {
    private boolean consumida = false;

    @Override
    public boolean usar(Personagem alvo) {
        if (consumida) {
            return false;
        }
        alvo.recuperarVida(20);
        consumida = true;
        return true;
    }

    @Override
    public String descrever() {
        if (consumida) {
            return "Poção vazia";
        }
        return "Poção cheia";
    }
}
```